CUET PG

DAILY CLASS NOTES

Computer Science

Digital Fundamental

Lecture – 12

Combinational Circuit

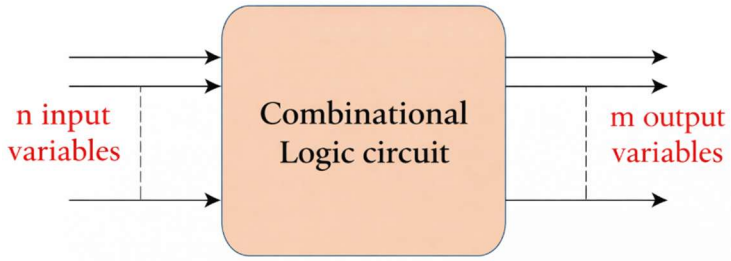


Combinational Circuit

Content: Combinational circuit, Adder, Subtractor, Multiplexer, DeMultiplexer, Encoder, and Decoder.

Combinational Circuit:

- ★ **A combinational circuit is a digital circuit where the output depends only on the current inputs**, not on any past inputs.
- ★ **There is no memory** in a combinational circuit, so it cannot store previous values.
- ★ **Output = Function of present inputs only** (no effect of history or previous input sequence).
- ★ Combinational circuits are built using **logic gates** like AND, OR, NOT, NAND, NOR, XOR, etc.
- ★ **No feedback path** exists in combinational circuits.
- ★ The **current state of the circuit is independent of the previous state**.
- ★ Any number of inputs (**n**) and any number of outputs (**m**) can be used.
- ★ Examples of combinational circuits: **Adders, Subtractors, Multiplexers, Demultiplexers, Encoders, Decoders**.



Block diagram of a combinational circuit

The **n input variable** comes from the external source, while the **m output variable** goes to the external destination. In many applications, the source or destinations are storage registers.

Types of Combinational Circuits

1. **Decoder**
   - ★ A decoder converts **binary code into a human-readable form**.
   - ★ It takes an *n-bit binary input* and activates **only one output line** out of $\mathbf{2^n}$
   - ★ Example: A decoder converts a **4-bit binary number** into signals for a **7-segment display**.
2. **Encoder**
   - ★ An encoder performs the **reverse of a decoder**.
   - ★ It converts **multiple input signals** into a smaller number of **binary outputs**.
   - ★ Example: It converts **7-segment display signals** back into a **4-bit binary code**.
3. **Multiplexer (MUX)**
   - ★ A multiplexer is a *data selector*.
   - ★ It selects **one input out of many inputs** and sends it to the single output line.
   - ★ Example: A **4-to-1 MUX** takes four inputs and outputs **only one** based on select lines..
4. **Demultiplexwer(DEMUX)**
   - ★ **Demultiplexer (DEMUX) takes one input signal and sends it to one selected output line.**
   - ★ **Select lines decide** which output will receive the input.
   - ★ It works like a **data distributor**, routing one input to many outputs.
   - ★ Example: A **1-to-4 DEMUX** sends input **I** to one of the four outputs (Y0, Y1, Y2, Y3) depending on the select lines **S1 and S0**.
   - ★ If S1S0 = 10 → Input goes to output Y2.

Half Adder vs Full Adder

Half Adder

1. Adds two 1-bit inputs (A and B).
2. Does NOT handle carry-in from a previous stage.
3. Produces two outputs:
    - Sum (S)
    - Carry (Cout)
4. It cannot be used for multi-bit addition directly.
5. Uses XOR (for Sum) and AND (for Carry) gates.

Full Adder

1. Adds three 1-bit inputs (A, B, and Carry-in).
2. Handles carry-in, so it supports multi-bit binary addition.
3. Produces two outputs:
    - Sum (S)
    - Carry-out (Cout)
4. Used in building ripple-carry adders and arithmetic circuits.
5. Made using two half-adders + OR gate (or equivalent design).

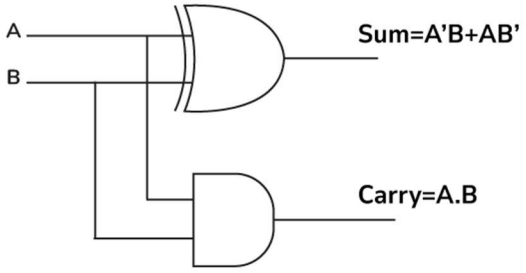
Main Difference (Simple statement)

- Half Adder: Adds only current bits. No carry-in.
- Full Adder: Adds current bits + previous carry (carry-in).

Logical Expression of Half Adder

Sum (S) = $A \oplus B$

Carry (C) = $A \cdot B$

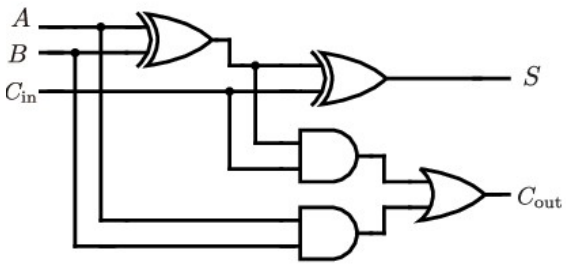


Truth Table			
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Logical Expression of Full Adder

CARRY-OUT = AB + BCin + ACin

SUM = (A ⊕ B) ⊕ Cin



Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

1. **Half Subtractor**

A **Half Subtractor** is a combinational circuit that subtracts **two 1-bit numbers** (A − B).

It produces:

- ★ **Difference (D)**
- ★ **Borrow (B_out)**

It **cannot** subtract from a previous borrow.

Half Subtractor – Truth Table

A	B	Difference (D)	Borrow (B_out)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Equations

- ★ **Difference D = A ⊕ B**
- ★ **Borrow B_out = A' · B**

2. **Full Subtractor**

A **Full Subtractor** subtracts **three 1-bit inputs**:

- ★ A
- ★ B
- ★ Borrow-in (**B_in**)

It produces:

- ★ **Difference (D)**
- ★ **Borrow-out (B_out)**

It can handle multi-bit subtraction because it includes **Borrow-in**.

Full Subtractor – Truth Table

A	B	B_in	Difference (D)	Borrow-out (B_out)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Equations

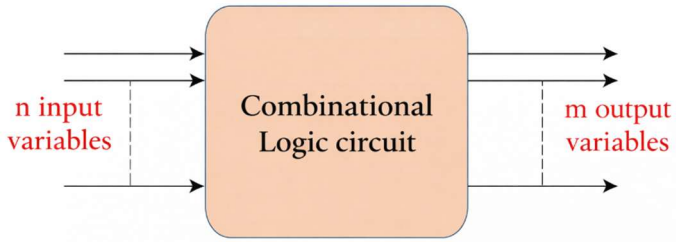
- Difference $D = A \oplus B \oplus B_in$
- Borrow-out $B_out = B \cdot A' + B_in \cdot (A \oplus B)'$

संयोजन सर्किट

विषयवस्तु: कॉम्बिनेशन सर्किट, एडर, सबट्रैक्टर, मल्टीप्लेक्सर, डीमल्टीप्लेक्सर, एनकोडर और डिकोडर।

कॉम्बिनेशन सर्किट:

★ एक कॉम्बिनेशन सर्किट एक डिजिटल परिपथ होता है जिसका आउटपुट केवल वर्तमान इनपुट पर निर्भर करता है, पिछले इनपुट पर नहीं।
★ एक कॉम्बिनेशन सर्किट में कोई मेमोरी नहीं होती, इसलिए यह पिछले मानों को संग्रहित नहीं कर सकता।
★ आउटपुट केवल वर्तमान इनपुट का फलन होता है (इतिहास या पिछले इनपुट अनुक्रम का कोई प्रभाव नहीं होता)।
★ कॉम्बिनेशन सर्किट AND, OR, NOT, NAND, NOR, XOR आदि जैसे लॉजिक गेट्स का उपयोग करके बनाए जाते हैं।
★ कॉम्बिनेशन सर्किट में कोई फीडबैक पथ नहीं होता।
★ पिपथ की वर्तमान स्थिति पिछली स्थिति से स्वतंत्र होती है।
★ किसी भी संख्या में इनपुट (n) और किसी भी संख्या में आउटपुट (m) का उपयोग किया जा सकता है।
★ कॉम्बिनेशन सर्किट के उदाहरण: एडर, सबट्रैक्टर, मल्टीप्लेक्सर, डीमल्टीप्लेक्सर, एनकोडर, डिकोडर।



Block diagram of a combinational circuit

इनपुट वेरिएबल n बाहरी स्रोत से आता है, जबकि **आउटपुट वेरिएबल m** बाहरी गंतव्य तक जाता है। कई अनुप्रयोगों में, स्रोत या गंतव्य स्टोरेज रजिस्टर होते हैं।

कॉम्बिनेशन सर्किट के प्रकार

1. **डिकोडर**
   ★ एक **डिकोडर बाइनरी कोड को मानव-पठनीय** रूप में परिवर्तित करता है।
   ★ यह **n-बिट बाइनरी इनपुट लेता है और 2n आउटपुट लाइनों** में से केवल एक को सक्रिय करता है।
   ★ उदाहरण: एक **डिकोडर 4-बिट बाइनरी संख्या को 7-सेगमेंट डिस्प्ले** के लिए सिग्नल में परिवर्तित करता है।
2. **एनकोडर**
   ★ **एनकोडर, डिकोडर के विपरीत कार्य** करता है।
   ★ यह कई **इनपुट सिग्नलों को कम संख्या में बाइनरी आउटपुट** में परिवर्तित करता है।
   ★ उदाहरण: यह **7-सेगमेंट डिस्प्ले सिग्नलों को वापस 4-बिट बाइनरी कोड** में परिवर्तित करता है।
3. **मल्टीप्लेक्सर (MUX)**
   ★ **मल्टीप्लेक्सर** एक डेटा सेलेक्टर होता है।
   ★ यह कई इनपुट में से एक इनपुट का चयन करता है और उसे एक आउटपुट लाइन पर भेजता है।
   ★ उदाहरण: एक **4-टू-1 MUX चार इनपुट** लेता है और **सेलेक्ट लाइनों के आधार** पर केवल एक आउटपुट देता है।
4. **डीमल्टीप्लेक्सर (DEMUX)**
   ★ **डीमल्टीप्लेक्सर (DEMUX) एक** इनपुट सिग्नल लेता है और उसे एक चयनित आउटपुट लाइन पर भेजता है।
   ★ चयनित लाइनें यह निर्धारित करती हैं कि कौन सा आउटपुट इनपुट प्राप्त करेगा।
   ★ यह एक डेटा वितरक की तरह काम करता है, जो एक **इनपुट को कई आउटपुट** तक पहुंचाता है।
   ★ उदाहरण: एक 1-से-4 DEMUX, चयनित लाइनों S1 और S0 के आधार पर इनपुट I को चार आउटपुट (Y0, Y1, Y2, Y3) में से किसी एक पर भेजता है।
   ★ यदि S1S0 = 10 → इनपुट आउटपुट Y2 पर जाता है।

हाफ एडर बनाम फुल एडर

हाफ एडर

1. यह दो 1-बिट इनपुट (A और B) को जोड़ता है।
2. यह पिछले चरण से आने वाले कैरी-इन को हैंडल नहीं करता है।
3. यह दो आउटपुट देता है:
   - सम (S)
   - कैरी (Cout)
4. इसका उपयोग सीधे मल्टी-बिट जोड़ के लिए नहीं किया जा सकता है।
5. यह XOR (योग के लिए) और AND (कैरी के लिए) गेट का उपयोग करता है।

फुल एडर

1. यह तीन 1-बिट इनपुट (A, B और कैरी-इन) को जोड़ता है।
2. यह कैरी-इन को संभालता है, इसलिए यह मल्टी-बिट बाइनरी जोड़ का समर्थन करता है।
3. यह दो आउटपुट देता है:
   - सम (S)
   - कैरी-आउट (Cout)
4. इसका उपयोग रिपल-कैरी एडर और अंकगणितीय सर्किट बनाने में किया जाता है।
5. इसे दो हाफ-एडर + OR गेट (या समकक्ष डिज़ाइन) का उपयोग करके बनाया जाता है।

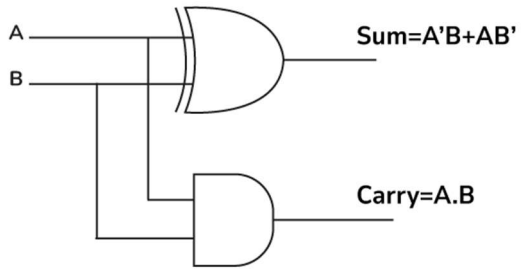
मुख्य अंतर (सरल कथन)

- हाफ एडर: केवल वर्तमान बिट्स को जोड़ता है। कोई कैरी-इन नहीं।
- फुल एडर: वर्तमान बिट्स + पिछले कैरी (कैरी-इन) को जोड़ता है।

हाफ एडर की तार्किक अभिव्यक्ति

Sum (S) = A ⊕ B

Carry (C) = A . B

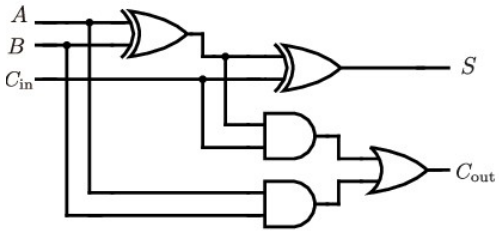


Truth Table			
A	**B**	**S**	**C**
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

फुल एडर की तार्किक अभिव्यक्ति

CARRY-OUT = AB + BCin + ACin

SUM = (A ⊕ B) ⊕ Cin



Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

1. हाफ सब्सट्रेक्टर

हाफ सबट्रैक्टर एक संयोजनात्मक परिपथ है जो **दो 1-बिट संख्याओं** (A − B) को घटाता है।

यह निम्न परिणाम देता है:

★ **डिफरेंस (D)**

★ **बॉरो (B_out)**

यह पिछले उधार से घटाया नहीं जा सकता।

हाफ सबट्रैक्टर – ट्रुथ टेबल

A	B	डिफरेंस (D)	बॉरो (B_out)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

समीकरण

★ **Difference D = A ⊕ B**

★ **Borrow B_out = A' · B**

2. फुल एडर

एक **फुल सबट्रैक्टर तीन 1-बिट इनपुट** को घटाता है:

★ A

★ B

★ Borrow-in (**B_in**)

वह प्रदान करता है:

★ **Difference (D)**

★ **Borrow-out (B_out)**

इसमें बरो-इन सुविधा शामिल होने के कारण यह **मल्टी-बिट घटाव** को संभाल सकता है।

फुल सबट्रैक्टर- ट्रुथ टेबल

A	B	B_in	Difference (D)	Borrow-out (B_out)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

समीकरण

- Difference D = A ⊕ B ⊕ B_in
- BORrow-out B_out = B·A' + B_in·(A ⊕ B)'

